

DATA STRUCTURES AND ALGORITHMS

COLLECTIONS

Collections is a built-in Python module that implements specialized container types, providing alternatives to Python's general-purpose built-in containers, such as lists, sets, and tuples.

List

★ A collection that is ordered and changeable is written within square brackets [].

★ Index starts with 0, items are separated with a comma

★ Include single/double quotation marks if the item is a string

★ **Syntax:** var_list = [item1, item2, item3,]

Basic Operation of a List

Access the item in the list - Access the list items by referring to the index number:

```
mylist = ['sun', 'moon', 'star', 'cloud']  
  
# print the 3rd element  
print(mylist[2])
```

Access range of items - you need to specify the range of indexes by selecting where to start and where to end the range. The return value will be a new list in the specified items

★ **Syntax:**
print(var_list[start_index:end_index])

Accessing a range of items without a start or end index.

★ **Syntax:** print (var_list[: end_index])

★ **Syntax:** print (var_list[start_index:]) range will go on to the end of the list

Negative Indexing - means beginning from the end, means -1 refers to the last item of the list.

Change Item Value - refers to the index number to change the value of a specific item.

★ **Syntax:** Additem[index] = new_value

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sky']  
  
mylist[3] = 'rain'  
  
# print all items from the list  
print(mylist)
```

Iteration within a list - use a loop to access/add items in the list.

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sky']  
  
for x in mylist:  
    print(x)
```

Check if an item exists in the list -

Use the in keyword to determine if a specified item is present in the list.

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sky']  
  
if 'moon' in mylist:  
    print('Yes, moon is in the list.')
```

Check the length of the list - use the len function to determine how many items a list has.

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sky']  
  
print(len(mylist))
```

Add item in the list - use the insert method to add an item at the specified index.

★ **append()** - adds exactly one item to the very end.

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sky']  
  
mylist.insert(1, 'rain')  
print(mylist)
```

Remove an item in the list -

★ To remove the specified item use the **remove()** method.

- ☆ To remove the item in the specified index or the last item if index is not specified use the **pop()** method
- ☆ To remove an item in the specified index use the **del** keyword.
- ☆ The **clear()** method will empty the list.

Count an item in the list - use the **count()** function to return the number of times an item appears in the list.

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sun']
print(mylist.count('sun'))
```

Reverse an item in the list - use reverse method to return the items of the list in reverse order

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sun']
mylist.reverse()
print(mylist)
```

Sort an item in the list - use sort method to return the items of the list in ascending order.

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sun']
mylist.sort()
print(mylist)
```

Extend an item in the list - use **extend()** method to add the elements of a list to the end of the current list

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sun']
newlist = ['rain', 'snow']
mylist.extend(newlist)
print(mylist)
```

Copy the entire item(s) in the list - use **copy()** method to copy the list.

```
mylist = ['sun', 'moon', 'star', 'cloud', 'sun']
mylist_copy = mylist.copy()
print(mylist_copy)
```

Tuples

- ☆ a collection which is ordered and unchangeable.
- ☆ are written with round () brackets
- ☆ **Syntax of a Tuple:** var_tuple = (item1, item2, item3,)

Properties of a Tuple

Access item in the tuple

```
sam_tuple=("summer","winter","fall","spring")
print(sam_tuple[2])
```

Index of an item in the tuple

```
sam_tuple=("summer","winter","fall","spring","rainy")
x=sam_tuple.index("winter")
print(x)
```

Range of Items

```
sam_tuple=("summer","winter","fall","spring","rainy")
print(sam_tuple[1:3])
```

```
sam_tuple=(5,8,2,6,4,1,3,7,9)
print(sam_tuple[:5])
```

```
sam_tuple=(5,8,2,6,4,1,3,7,9)
print(sam_tuple[4:])
```

Negative Indexing

```
sam_tuple=("summer","winter","fall","spring","rainy")
print(sam_tuple[-2])
```

Range of Negative Indexing

```
sam_tuple=("summer","winter","fall","spring","rainy")
print(sam_tuple[-4:-2])
```

Iteration with a tuple

```
sam_tuple=("summer","winter","fall","spring","rainy")
for x in sam_tuple:
    print(x)
```

in method

```
sam_tuple=("summer","winter","fall","spring","rainy")
if "winter" in sam_tuple:
    print(Yes)
```

Length of the tuple

```
sam_tuple=("summer","winter","fall","spring","rainy")
print(len(sam_tuple))
```

Note: Tuples are unchangeable. You cannot add items to it once the tuple is created.

Delete a tuple - use the **del** keyword to delete the tuple completely.

```
sam_tuple=("summer","winter","fall","spring","rainy")
del sam_tuple
print(sam_tuple)
```

Note: You cannot remove items in the tuple but you can delete the tuple completely.

Count an item in the tuple

```
sam_tuple=(5,8,2,6,4,1,3,7,9)
x=sam_tuple.count(8)
print(x)
```

Join a tuple - use the plus (+) operator to join two or more tuples.

```
sam_tuple1=(5,8,2,6,4)
sam_tuple2=(1,5,3,7)
x=sam_tuple1+sam_tuple2
print(x)
```

Sets

★ a collection which is unordered and unindexed.

★ are written with curly {} brackets

★ **Syntax of a Sets:** var_sets = {item1, item2,}

Note: the set items are unordered, meaning the items will appear in a random order.

Properties of a Sets

Access items in the sets - you cannot access items in a set by referring to an index. An iteration can be used through the set of items or use in keyword to specify an item in the sets.

```
sam_sets={"summer","winter","fall","spring"}
for x in sam_sets:
    print(x)
```

Add an item in the sets - use the **add()** method to add one item to the set.

```
sam_sets={"summer","winter","fall","spring"}
sam_sets.add("rainy")
```

Add multiple items in the sets - use the **update()** method to add multiple items to the set. Enclosed the items in the [] to exclude any duplicate items

```
sam_sets={"summer","winter","fall","spring"}
sam_sets.update(["sun","fall","moon","winter"])
print(sam_sets)
```

Join two sets - use the **union()** method to return a new set with all items from both sets (exclude the duplicates).

```
sam_sets1={"summer","winter","fall","spring"}
sam_sets2={"sun","winter"}
sam_set3=sam_sets1.union(sam_sets2)
print(sam_set3)
```

Discard an item in the sets - use the **discard()** method to remove the specified item from both sets.

```
sam_sets={"summer","winter","fall","spring"}
sam_sets.discard("winter")
print(sam_sets)
```

Difference of items in the sets - use the **difference()** method to return a set that contains the items that only exist in set x and not in set y.

```
sam_sets1={"summer","winter","fall","spring"}
sam_sets2={"summer","winter"}
x=sam_sets1.difference(sam_sets2)
print(x)
```

Intersection of an item in the sets - use the **intersection()** method to return

a set that contains the items that exist in both set x and in set y.

```
sam_sets1={"summer","winter","fall","spring"}
sam_sets2={"summer","winter"}
x=sam_sets1.intersection(sam_set2)
print(x)
```

Length - return the number of items

```
sam_sets={5,8,2,6,4,1,3,7,9}
print(len(sam_sets))
```

Remove - remove item; if the item to be removed does not exist, it will display an error message (when using the pop method, you will not know which item gets removed).

```
sam_sets={5,8,2,6,4,1,3,7,9}
sam_sets.remove(6)
print(sam_sets)
```

Dictionary (dict)

★ **Syntax:** var_dict = {key1: value1, key2: value2,}

Note: Each key must be unique, but multiple keys can have the same value.

★ known as a map, hash map, or associative array and is a collection of key-value pairs.

★ Unlike lists or arrays, dictionaries allow you to retrieve or modify values using a unique key.

★ They are used when data needs to be organized for quick access, lookup, or categorization

Characteristics:

- ★ Each key maps to specific value
- ★ use the key to access or modify its associated value.
- ★ changeable (mutable)
- ★ Indexed by keys and not by numeric position

```
File object = { "key1": value1, "key2": value2, "keyn": valuen}

OR

Fileobject = {
    "key1": value1,
    "key2": value2,
    "keyn": valuen,
}
```

Properties of a dictionary Declare/Create and display

```
# Create a new dictionary
dict_file = {
    "stunum": 12345,
    "studname": "Peter",
    "grade": 85,
    "remark": "Passed"
}

# Display the contents of a dictionary
print(dict_file)
```

Output:
({"stunum": 12345, "studname": "Peter", "grade": 85, "remark": "Passed"})

```
# Create a new dictionary
dict_file = {
    1: 100,
    2: 300,
    3: 500,
    4: 700
}

# Display the contents of a dictionary
print(dict_file)
```

Output:
{1: 100, 2: 300, 3: 500, 4: 700}

Access an item in a dictionary using the get () method—

```
# Create a new dictionary
dict_file = {
    "stunum": 12345,
    "studname": "Peter",
    "grade": 85,
    "remark": "Passed"
}

# Display the items of a dictionary
Name=dict_file.get(studname)
print(Name)
```

Output:
Peter

Add an item in the dictionary

```
# Create a new dictionary
dict_file = {
    1: 100,
    2: 300,
    3: 500,
    4: 700
}

# Add an item
dict_file[5]= 900
dict_file["code"] = "Old"
print(dict_file)
```

Output:
{1: 100, 2: 300, 3: 500, 4: 700, 5: 900, "code": "red"}

Add an item in the dictionary using input function

```
# Create a new dictionary
dict_file = {
    1: 100,
    2: 300,
    3: 500,
    4: 700
}

# Add an item using the input function
keyname=input("Enter new key name") # user input is class
keyvalue=val (input("Enter value for the new key name")) # user input is 20
dict_file[keyname]=keyvalue
print(dict_file)
```

Output:
{1: 100, 2: 300, 3: 500, 4: 700, 5: 900, "class": 20}

Iterate items in the dictionary

★ Iterating a dictionary will return the keys of the dictionary, but there are methods to return the values.

★ While **values()** methods can also return the values of an item, the **items()** method will iterate over the key and the values of an item.

```
# Create a new dictionary
dict_file = {
    "stunum": 12345,
    "studname": "Peter",
    "grade": 85,
    "remark": "Passed"
}

# print all the values
for x in dict_file:
    print(dict_file[x])
```

Output:

```
12345
Peter
85
Passed
```

```
# Create a new dictionary
dict_file = {
    "stunum": 12345,
    "studname": "Peter",
    "grade": 85,
    "remark": "Passed"
}

# print all the key names
for x in dict_file:
    print(x)
```

Output:

```
stunum
studname
grade
remark
```

Change the value of an item in a dictionary

```
# Create a new dictionary
dict_file = {
    "stunum": 12345,
    "studname": "Peter",
    "grade": 85,
    "remark": "Passed"
}

# change the value of item
dict_file["grade"] = 70
print(dict_file)
```

Output:

```
{'stunum': 12345, 'studname': 'Peter', 'grade': 70, 'remark': 'Passed'}
```

Check if an item exists in the dictionary - use the `in ()` keyword

```
# Create a new dictionary
dict_file = {
    "stunum": 12345,
    "studname": "Peter",
    "grade": 85,
    "remark": "Passed"
}

if "Peter" in dict_file:
    print("Exists")
else:
    print("Does not exists")
```

Output:

```
Exists
```

Copy the items of a dictionary - use the `copy()` and `dict()` method to copy the entire contents of a dictionary

```
# Create a new dictionary
dict_file = {
    "stunum": 12345,
    "studname": "Peter",
    "grade": 85,
    "remark": "Passed"
}

# copy the contents of a dictionary using the copy()
copy_file = dict_file.copy()
print(copy_file)
```

Output:

```
{'stunum': 12345, 'studname': 'Peter', 'grade': 85, 'remark': 'Passed'}
```

```
# copy the contents of a dictionary using the dict()
new_file = dict(dict_file)
print(new_file)
```

Remove an item in the dictionary

- ★ `pop()` - removes an item with the provided key name
- ★ `popitem()` - remove and return an arbitrary (key, value) item pair
- ★ `clear()` - removed all items at once
- ★ `del` -remove individual items or the entire dictionary

```
# Create a new dictionary
dict_file = {
    "stunum": 12345,
    "studname": "Peter",
    "grade": 85,
    "remark": "Passed"
}

# remove a particular item and returns its value
print(dict_file.pop("stunum"))
```

Output:

```
12345
('remark', 'Passed')
('studname', 'Peter', 'grade': 70)
```

```
# remove the last item and returns the value
print(dict_file.popitem())
print(dict_file)
```

```
# Create a new dictionary
dict_file = {
    1: 100,
    2: 300,
    3: 500,
    4: 700
}

# remove all items
dict_file.clear()

# delete the dictionary
del dict_file
```

Output:

```
()
Traceback (most recent call last):
  File "<string>", line 30, in <module>
NameError: name dict_file is not defined
```

Multiple dictionary

- ★ A collection of dictionaries—either stored inside another dictionary or inside a list
- ★ allows you to represent complex structured data

```
# Create a new dictionary
dict_file1 = { "code": 8063, "size": 20 }
dict_file 2= { "code": 8020, "size": 30 }
dict_file 3= { "code": 8030, "size": 40 }

all_file = {
    "file1": dict_file1,
    "file2": dict_file2,
    "file3": dict_file3
}

# print the contents of the dictionary
print(all_file)
```

Output:

```
{'file1': {'code': 8063, 'size': 20}, 'file2': {'code': 8020, 'size': 30}, 'file3': {'code': 8030, 'size': 40}}
```

FILE HANDLING

Sample Code to Write a New Text file

```
# Create a new Text File and save in drive C
Samplefile=open("c:\activity1.txt","w+")

Samplefile.write("Hello in line 1 ")
Samplefile.write("Hello in line 2 ")
Samplefile.write("Hello in line 3 ")

# close the file when done
samplefile.close()

# Check the text file ( C:\activity1.txt)
```

Check the text file (C:\activity1.txt)

```
Hello in line 1
Hello in line 2
Hello in line 3
```

Sample Code to Write a New Text file with an Iteration

```
# Create a new Text File and save in drive C
Samplefile=open("c:\activity1.txt","w+")

# write 5 hellos in a text file
for x in range(5):
    Samplefile.write("Hello in line %d\n" %(i+1))

# close the file when done
Samplefile.close()

# Check the text file ( C:\activity1.txt)
```

Check the text file (C:\activity1.txt)

```
Hello in line 1
Hello in line 2
Hello in line 3
Hello in line 4
Hello in line 5
```

Sample Code to Read a Text file

```
# Read a Text File saved in drive C
samplefile=open("c:\activity1.txt","r")

# print the contents of Samplefile
print(samplefile.read())

# close the file when done
samplefile.close()
```

Hello
Welcome to DSALGO class
Good luck.

Sample Code to Read a part of a Text file

```
# Read a Text File saved in drive C
samplefile=open("c:\activity1.txt","r")

#Returns the number of a specified character
print(samplefile.read(3))

# close the file when done
samplefile.close()
```

Output:
Hel

Sample Code to return 1 line using the readline()

```
# Read a Text File saved in drive C
samplefile=open("c:\activity1.txt","r")

#Returns the first line
print(samplefile.readline())

# close the file when done
samplefile.close()
```

Output:
Hello

Sample Code to return multiple line using the readline()

```
# Read a Text File saved in drive C
samplefile=open("c:\activity1.txt","r")

#Returns 2 lines
print(samplefile.readline())
print(samplefile.readline())

# close the file when done
samplefile.close()
```

Output:
Hello
Welcome to DSALGO class

Sample Code to return the whole line using the readline () and iteration

```
# Read a Text File saved in drive C
samplefile=open("c:\activity1.txt","r")

for x in samplefile:
    print(x)

# close the file when done
samplefile.close()
```

Output:
Hello
Welcome to DSALGO class
Good luck

Sample Code to append to an existing file

```
# Read a Text File saved in drive C
A=open("c:\activity1.txt","a")

A.write("I added another line")
# close the file when done
A.close()

# to check the content of activity1.txt
A=open("c:\activity1.txt","r")
print(A.read())
```

Output:
Hello
Welcome to DSALGO class
Good luck
I added another line

Sample Code to overwrite an existing file

```
# Read a Text File saved in drive C
A=open("c:\activity1.txt","w")

A.write("Opppps.. a new content")
# close the file when done
A.close()

# to check the content of activity1.txt
A=open("c:\activity1.txt","r")
print(A.read())
```

Output:
Opppps.. a new content

Remove/delete the text file - Import the os.remove() function from the OS module

```
import OS
os.remove("filename")

Example:
import os
os.remove("c:\activity1.txt")
```

Check the text file before you remove/delete

```
import OS
if os.path.exists("c:\activity1.txt"):
    os.remove("c:\activity1.txt")
else:
    print("File not exists")
```

"x" represents the Exclusive Creation mode. It opens a file specifically for writing, but only if the file does not already exist. If you attempt to use this mode on a filename that is already taken, Python will block the operation and raise a `FileExistsError`.

r+ opens an existing file for reading and writing without erasing it
w+ overwrites the file entirely by wiping its contents to zero length.

ONE DIMENSIONAL ARRAY

A linear collection of elements is stored in contiguous memory locations and accessed using a single index.

It is often described as a list of elements arranged in a single row, where each element can be identified by its position (or index).

Characteristics:

- ◆ All elements are of the same data
- ◆ The index starts at 0.
- ◆ Arrays allow fast access to any element using its index.
- ◆ Uses the array module from the standard library

Common Type codes of One-Dimensional Array

Type Code	Type	Size(Bytes)
i	int	2 or 4
f	float	4
d	float/double	8
u	unicode char	2

Properties of One-Dimensional Array

Create and declare an array

◆ **Syntax:** from array import array
var = array('typecode', [value1, value2, value3, ...])

```
# import array
import array as arr
a = arr.array('i', [1,2,3,4,5])

print(a)
```

Output:
array('i', [1, 2, 3, 4, 5])

Accessing element of an array

```
# import array
import array as arr
a = arr.array('i', [2,5,4,6,3,7,1,9])

print("First element:", a[0])
print("? element:", a[3])
print("Last element:", a[-1])
```

Output:
First element: 2
Second element: 6
Last element: 9

Note: The index starts from 0 (not 1) similar to lists

Slicing element of an array

```
import array as arr

arr_list = [5,25,10,3,14,6,20,7,15]
num_arr = arr.array('i', arr_list)

print(num_arr[1:6]) # 3rd to 6th
print(num_arr[:5]) # beginning to 5th
print(num_arr[3:]) # 4th to end
print(num_arr[:3]) # beginning to 3rd
```

Output:
array('i', [25, 10, 3, 14, 6])
array('i', [5, 25, 10, 3])
array('i', [3, 14, 6, 20, 7, 15])
array('i', [5, 25, 10, 3, 14, 6])

Adding elements to an array

◆ Use the **append()** method to add an element in the array

◆ Use the **extend()** method to add several elements in the array

```
import array as arr
num = arr.array('i', [2,5,3,4,1,6])

# add elements at the last
num.append(8)
print(num)

# extend() appends iterable to the end of the array
num.extend([1,2,3])
print(num)
```

Output:
array('i', [2, 5, 3, 4, 1, 6, 8])
array('i', [2, 5, 3, 4, 1, 6, 8, 1, 2, 3])

Adding elements of an array using the input function

```
# import array
num_array = array('i', [])
n = int(input("number of values to input")) // 5
for x in range(n):
    num = int(input("Enter values for array"))
    num_array.append(num)

print(num_array)
```

Remove/ delete an array element

◆ Use the **del** statement to delete one or more items from an array

◆ Use the **remove()** method to remove a given item

◆ Use **pop()** method to an item at the given index

```
import array as arr
num_array = arr.array('i', [2, 4, 5, 7, 3, 6])

# remove element at index 2 (third element)
del num_array[2]
print(num_array)
```

Output:
array('i', [2, 4, 7, 3, 6])

```
import array as arr
num_array = arr.array('i', [2, 4, 5, 7, 3, 6])

# delete the entire array
del num_array
print(num_array)
```

Output:
print(num_array) # Error: array is not defined
NameError: name 'num_array' is not defined

```
import array as arr
num_array = arr.array('i', [2, 4, 5, 7, 3, 6])

# return the value and remove item 7 from the array
num_array.remove(7)
print(num_array)
```

Output:
array('i', [2, 4, 5, 3, 6])

```
import array as arr
num_array = arr.array('i', [2, 4, 5, 7, 3, 6])

# return the value then remove item 7 from index
print(num_array.pop(4))
print(num_array)
```

Output:
3
array('i', [2, 4, 5, 7, 3, 6])

Access array elements

```
import array as arr
num_array = arr.array('i', [2, 4, 5, 7, 3, 6])
new_array = []

# get the sum of elements
X = num_array[0] + num_array[1]
new_array.append(X)
print(new_array)

# get the difference of index 3 and 4
X = num_array[3] - num_array[4]
new_array.append(X)
print(new_array)

# get the product of index 2 and 4
X = num_array[2] * num_array[4]
new_array.append(X)
print(new_array)
```

Output:
[6]
[6, 1]
[6, 1, 15]
[6, 1, 15, 1.6666666666666667]
[6, 1, 15, 1.6666666666666667, 1]
[6, 1, 15, 1.6666666666666667, 1, 4]

TWO DIMENSIONAL ARRAY

Is an array within an array. It organizes data into a grid or table format using rows and columns. In this structure, each data element is identified by two indices:

- ★ One for the row position
- ★ One for the column position

Characteristics:

- ★ represent matrices, tables or grids
- ★ each row is itself an array
- ★ accessed using array[row][column]
- ★ it uses a list collection

Properties of two-dimensional array

Declaration of two-dimensional array

```
#2D with 2 rows and 2 columns
var_name = [
    [10,5,20], [10,20,30]]

or

var_name = [
    [10,5,20],
    [10,20,30]]
```

Access two-dimensional array - specify the index referring to the main/parent and

another index referring to the position of the data element in the inner array.

TDim = [[3, 1, 5, 6], [2, 4, 9, 7], [3, 4, 6, 9], [5, 4, 6, 9]]

main	0	1	2	3
inner	0 1 2 3	0 1 2 3	0 1 2 3	0 1 2 3

TDim = [[3, 1, 5, 6], [2, 4, 9, 7], [3, 4, 6, 9], [5, 4, 6, 9]]

```
print(TDim[1]) # display all elements of main 1
print(TDim[2][1:3]) # display 2 elements from main 2
print(TDim[1][2]) # display element from main 1 inner 2
print(TDim[3][0]) # display element from main 3 inner 0
print(TDim[0][2]) # display element from main 0 inner 2
```

```
[2, 4, 9, 7]
[4, 6]
9
5
5
```

Display all elements in a two dimensional array

```
TDim = [[3,1,5,6], [2,4,9,7], [3,4,6,9], [5,4,6,9]]
for main in TDim:
    for inner in main:
        print(inner,end = " ")
    print()
```

```
3 1 5 6
2 4 9 7
3 4 6 9
5 4 6 9
```

Insert elements in a two dimensional array - use the insert method() to insert new data elements at specific position.

TDim = [[3,1,5,6], [2,4,9,7], [3,4,6,9], [5,4,6,9]]

TDim.insert(2,[2,2,2,2])

```
for main in TDim:
    for inner in main:
        print(inner,end = " ")
    print()
```

```
3 1 5 6
2 4 9 7
2 2 2 2
3 4 6 9
5 4 6 9
```

Delete elements in a two-dimensional array

TDim = [[3,1,5,6], [2,4,9,7], [3,4,6,9], [5,4,6,9]]

```
del TDim[3] # delete entire main 3
del TDim[0][0] # delete element from main 0 inner 0
```

```
for main in TDim:
    for inner in main:
        print(inner,end = " ")
    print()
```

```
1 5 6
2 4 9 7
3 4 6 9
```

Add elements in a two-dimensional array using a looping statement

```
main=Int(input("input"))
TDim = []
for row in range(main):
    TDim.append([int(inner) for inner in input().split()])
print(TDim)
```

Working with the elements of a two-dimensional array

TDim = [[3,1,5,6], [2,4,9,7], [3,4,6,9], [5,4,6,9]]

```
X=TDim[0][0] + TDim[1][2]
print(X)
X=TDim[2][1] + TDim[3][0]
print(X)
```

12

9

compute the sum of each main

```
s = 0
for main in range(len(TDim)):
    for inner in range(len(TDim[main])):
        s += TDim[main][inner]
```

15

22

22

24

print(s)

s=0 # reset value of s to 0

MAP

map() function is used to apply a given function to each item of an iterable collection (list, tuple, or set).

✧ returns a map object and convert it into a list to view the results

✧ It takes two arguments (a function to apply and one or more iterables)

✧ **Syntax:** Map (function, iterables)

Sending iterable objects in a function

```
def mymap(no,items):
```

```
    return no + qty # convert the value of no and qty
```

```
myval=map(mymap, ("one", "two", "three", "four"), ("paper", "pencil", "pen", "marker"))
```

```
# print map for readability using a list
print(list(myval))
```

Output:

```
['onepaper', 'twopencil', 'threepen', 'fourmarker']
```

Return values from map

```
def squareval(num):
    return num * num
```

```
myval= (2,4,3,1) # assign values using set
answer=map(squareval,myval)
print(answer)
```

Output:

```
16, 1, 9, 4
```

#to return the values the answer variable, you have to convert the map to a set

```
mapanswer=set(answer)
print(mapanswer)
```

#note Set is unordered indexed

Passing multiple iterables using lambda

```
val1=[ 2, 4, 6, 8]
```

```
val2=[1, 3, 5, 7]
```

Output:

```
2,12,30,56
```

```
result=map(lambda ans1, ans2: ans1 * ans2, val1,val2)
print(list(result))
```

#note a list is used for val1 and val2

